



Decentralized Voting System Smart Contract on Test Net

Details

Sepolia Test Network

Solidity

Remix IDE

MetaMask

Feature

Platform

Language

Environment

Wallet

Prepared by:

Shahin Zamanian

Student ID: C70309635

Course: BAN 5700

Instructor: Professor Hamidreza Ahady Dolatsara

Submission Date: 28 April 2026

GitHub Repository:

<https://github.com/shahinzamaniaan-max/Voting-System-Project-for-Blockchain-Course.git>

Worcester, Massachusetts

Spring 2026

1. Introduction

1.1 Problem Statement

Traditional voting systems often suffer from a lack of transparency, vulnerability to manipulation, and centralized control. These limitations reduce public trust in election outcomes and increase the risk of fraud, data tampering, and unauthorized modifications to voting records.

To address these challenges, this project proposes a decentralized voting system based on blockchain technology. In this system, votes are securely recorded on the blockchain, ensuring that once they are submitted, they cannot be altered or deleted. The use of smart contracts enables automated enforcement of voting rules, including the restriction that each participant is allowed to vote only once. This approach enhances transparency, improves security, and provides a tamper-proof mechanism for managing the voting process.

1.2 Research Question

Nowadays, blockchain technology is increasingly used to design secure, transparent, and tamper-proof systems across various domains. Its decentralized nature and immutable data structure make it particularly suitable for applications that require high levels of trust and integrity, such as voting systems. This study addresses the following research question: how can a smart contract-based voting system on the Ethereum test network ensure secure and transparent voting while preventing duplicate votes? To answer this question, a decentralized voting application is designed and implemented using Solidity smart contracts. The system leverages blockchain properties to ensure that all votes are securely recorded, cannot be modified after submission, and that each participant is allowed to vote only once.

1.3 Literature Review

Recent research in blockchain technology has explored the use of smart contracts for decentralized voting systems. Blockchain-based voting applications leverage key features such as immutability, transparency, and decentralized validation to ensure secure and trustworthy election processes.

In Ethereum-based systems, smart contracts are commonly used to manage voting logic, including candidate registration, vote casting, and result tracking. These systems often utilize mappings to prevent duplicate voting and ensure that each address can participate only once.

While some advanced platforms incorporate token-based governance and complex voting mechanisms, simpler implementations focus on core functionalities such as one-vote-per-user enforcement and transparent result storage. This project follows the latter approach by implementing a lightweight and reproducible voting system using Solidity, emphasizing clarity, security, and educational value.

Overall, existing studies highlight the effectiveness of blockchain in governance and voting applications. Platforms such as DAO-based systems, Aragon-style governance models, and Ethereum smart contract voting frameworks demonstrate different levels of complexity and scalability. Therefore, lightweight implementations like this project remain important for educational purposes and for demonstrating core blockchain principles in a clear and practical manner.

1.4 Gap Analysis

Existing blockchain-based voting solutions often focus on large-scale governance systems and complex token-based mechanisms. However, there is a lack of simple and reproducible implementations that clearly demonstrate fundamental voting logic using smart contracts.

This project addresses this gap by developing a lightweight decentralized voting system, focusing on core features such as one-vote-per-address enforcement, transparent vote counting, and secure data storage.

1.5 Relevance to BAN 5700

This project aligns with BAN 5700 by applying blockchain concepts, smart contract development, and decentralized system design to solve a real-world problem. It demonstrates practical implementation of blockchain analytics and secure data management.

2. Technical Design & Architecture

2.1 Platform Selection

Platform Selection

Ethereum was selected due to its mature ecosystem, strong developer support, and robust smart contract capabilities. The Sepolia test network was used to simulate real blockchain behavior without financial risk.

In addition, using Ethereum with Remix IDE provided a fully online development environment, eliminating the need for complex local setup or large software installations. Compared to some alternative platforms, this approach avoided compatibility issues and reduced system resource usage, making the development process more accessible and efficient.

2.2 System Architecture

The system consists of three main components:

- MetaMask Wallet (user interaction and transaction signing)
- Remix IDE (development and deployment)
- Ethereum Sepolia Network (execution and storage)

Architecture Flow:

User → MetaMask → Smart Contract → Remix (Compile and Deploy) →Blockchain

This architecture ensures a clear separation of responsibilities between user interaction, development environment, and blockchain execution. It also enables secure transaction handling through MetaMask and reliable state persistence on the Ethereum test network. Overall, the design supports transparency, modularity, and ease of deployment for decentralized applications.



2.3 Smart Contract Design

The voting system is implemented using Solidity and includes the following components:

- Candidate structure (name, vote count)
- Mapping for vote tracking
- Dynamic array for candidates
- Owner-based access control

The voting system is implemented as a Solidity smart contract that defines the core logic and data structures required for secure and transparent voting.

The contract consists of several key components. First, a Candidate structure is defined to store each candidate's name and the number of votes received. This allows organized and efficient data management. Second, a mapping is used to track whether a wallet address has already voted. This mechanism enforces the one-vote-per-address rule and prevents duplicate voting.

Third, a dynamic array is used to store all candidates, allowing flexible addition of new candidates during contract execution.

Finally, an owner-based access control mechanism is implemented to restrict certain operations, such as adding new candidates, ensuring that only authorized users can modify critical parts of the system.

2.4 Core Logic

The contract follows a structured workflow:

1. Initialize candidates
2. Allow owner to add candidates
3. Allow users to vote
4. Prevent duplicate voting
5. Provide result retrieval

The smart contract follows a structured and sequential workflow to ensure correct and secure execution of the voting process.

First, the system is initialized through the constructor, where an initial list of candidates is provided and stored on the blockchain. Second, the contract allows the owner to add new candidates dynamically. This ensures flexibility while maintaining controlled access to critical operations. Third, users can cast their votes by selecting a candidate using their index. Each vote is recorded on the blockchain as a transaction. Fourth, the system enforces a one-vote-per-address rule using a mapping mechanism. This prevents duplicate voting and ensures fairness in the election process. Finally, the contract provides read-only functions that allow users to retrieve the number of candidates and view voting results at any time without modifying the blockchain state.

2.5 Security Considerations

Although this system provides fundamental security features, further enhancements such as identity verification, front-end validation, and protection against front-running attacks can be considered in future development.

The system incorporates multiple security mechanisms:

- Access control using owner restriction
- Prevention of double voting using mapping
- Input validation using require statements
- Immutable data storage on blockchain

Additionally, the system relies on blockchain transparency to ensure that all voting activities are publicly verifiable and auditable. The use of smart contracts eliminates the need for centralized intermediaries, reducing the risk of single-point failure. Furthermore, event logging mechanisms can be introduced in future versions to improve traceability of voting actions.

Scalability improvements may also be required for handling larger numbers of participants in real-world deployments. Finally, integrating decentralized identity (DID) solutions could further strengthen voter authentication and system reliability.

Additionally, the system enforces the one-vote-per-user rule by tracking wallet addresses through a mapping structure, ensuring that each address can cast only a single vote.

The voting process is executed through the `vote()` function, which validates candidate index inputs before recording any transaction. Candidate data, including name and vote count, is stored in a dynamic array, allowing real-time updates after each vote. The owner-based control mechanism

restricts access to the `addCandidate()` function, preventing unauthorized modification of candidate lists. All voting records are permanently stored on the Ethereum Sepolia test network, ensuring transparency and immutability of election results.

3. Implementation & Results

3.1 Development Process

The development of the smart contract was carried out using an iterative and structured approach. First, the voting logic was designed conceptually, including candidate management, vote casting, and vote restriction rules. Next, the smart contract was implemented in Solidity using Remix IDE as the development environment. The contract code was written, compiled, and debugged within Remix to ensure correctness and functionality.

MetaMask was then used to connect the development environment to the Ethereum Sepolia test network. It served as the wallet for managing accounts, signing transactions, and paying gas fees using test ETH.

After successful compilation, the contract was deployed to the Sepolia network using the Injected Provider feature in Remix. Each transaction, including deployment and voting actions, was confirmed through MetaMask.

Finally, the system was tested through multiple scenarios, including successful voting, duplicate vote prevention, and result retrieval, to validate the correctness and reliability of the implementation.

3.2 Deployment Steps

1. Install MetaMask
2. Connect to Sepolia network
3. Obtain test ETH from faucet
4. Open Remix IDE
5. Compile the smart contract
6. Deploy using MetaMask
7. Confirm transaction

3.3 Coding

The constructor runs only once during deployment. Before implementing the functions, the system behavior was defined as follows: Users interact with the contract by sending transactions, which either modify the blockchain state (like voting) or read data (like checking results).

`constructor(string[] memory candidateNames) .` Its purpose is:

- assigning contract owner
- receiving initial candidate list
- storing initial candidates

Example deployed candidates:

- Ali
- Reza
- Sara

The constructor is a special function in the smart contract that is executed only once during the deployment phase. It plays a crucial role in initializing the initial state of the system on the blockchain before any user interaction begins. In this project, the constructor is defined using the Solidity syntax `constructor(string[] memory candidateNames)` and is responsible for setting up the foundational data required for the voting system.

Before implementing the main functions, the overall system behavior was designed in a way that users interact with the contract through transactions. These transactions either modify the blockchain state, such as casting a vote, or retrieve stored data, such as viewing candidate results. This separation between state-changing and read-only operations ensures clarity and efficiency in the system's execution.

The main purpose of the constructor is threefold. First, it assigns the deployer of the contract as the owner, enabling controlled access to administrative functions such as adding new candidates. Second, it receives an initial list of candidate names as input during deployment. Third, it stores these candidates in a dynamic array structure on the blockchain, making them available for voting immediately after deployment.

For example, during deployment, the system can be initialized with candidates such as Ali, Reza, and Sara. These initial entries are stored permanently on-chain and serve as the starting point for the voting process. This design ensures that the system is immediately functional after deployment without requiring additional manual configuration.

Main Functions:

- `addCandidate()`

The `addCandidate()` function allows only the owner to add new candidates to the system, ensuring controlled access. `function addCandidate(string memory name)`. This function allows the owner to add a new candidate.

The `addCandidate()` function allows only the contract owner to add new candidates to the system, ensuring controlled access and preventing unauthorized modifications. This function enhances the security and integrity of the voting process. It is designed to maintain proper governance over candidate registration. The function takes a string input representing the candidate's name.

Example:

```
addCandidate("Hamid")
```

Security condition: `require(msg.sender == owner)`

Only the owner can execute this function.

- **vote()**

The `vote()` function is the core of the system. It allows users to vote for a candidate by index while enforcing rules such as one vote per address and valid candidate selection. `function vote(uint candidateIndex) .` This is the core function of the project.

The `vote()` function is the core functionality of the system, enabling users to cast their votes by selecting a candidate index. It ensures that each vote is properly recorded on the blockchain and permanently stored. The function also enforces validation checks to maintain system correctness. Additionally, it prevents invalid or duplicate voting actions.

Its responsibilities are:

- checking whether the user has already voted
- validating candidate index
- recording the vote
- incrementing vote count

Example:

`vote(0)`

This means voting for candidate index 0.

For example:

0 → Ali
1 → Reza
2 → Sara
3 → Hamid

- **Vote Restriction Logic**

Getter functions such as `getCandidate()` and `getCandidatesCount()` are used to retrieve voting results from the blockchain without modifying the state. The following line prevents multiple voting: `require(!hasVoted[msg.sender], "You already voted") .`

The vote restriction mechanism ensures that each wallet address can participate only once in the voting process. This is enforced using a mapping structure that tracks voting status for each user. The condition `require(!hasVoted[msg.sender])` prevents double voting attempts. This logic guarantees fairness and integrity in the election system.

This means:

each wallet address can vote only once in the entire system. This is a global one-vote-per-address rule. If user add vote more than once a message “you have already voted” appears on the screen.

- Getter Functions

Getter functions such as `getCandidate()` and `getCandidatesCount()` provide read-only access to blockchain data without modifying the state. These functions allow users to retrieve real-time voting results in a transparent manner. They improve usability by enabling external systems to query contract data easily. The output reflects the current state of candidates and their vote counts.

Total candidates

```
getCandidatesCount()
```

Returns total number of candidates.

Candidate result

```
getCandidate(index)
```

Returns:

- candidate name
- vote count

Example output:

```
("Ali", 1)
```

3.4 Testing and Validation

The following test cases were executed to validate the functionality and reliability of the smart contract deployed on the Ethereum Sepolia test network. The testing phase focused on verifying the core operations of the system, including vote execution, prevention of duplicate voting, and accurate retrieval of candidate data. Each function was tested under controlled scenarios to ensure that the contract behaves as expected under different conditions. The results confirmed that all implemented features operate correctly, and the smart contract successfully maintains data integrity, enforces security rules, and provides consistent outputs across all tested cases.

Test Case	Input	Expected Result	Outcome
Vote Execution	<code>vote(0)</code>	Transaction successful and vote recorded	Passed
Duplicate Voting	<code>vote(1)</code> (same address)	Transaction reverted with error message	Passed
Candidate Query	<code>getCandidate(0)</code>	Correct candidate name and updated vote count	Passed

Test 1 – Successful Vote

Input:

`vote(0)`

Output:

Transaction mined and execution completed

This confirms successful transaction execution.

The first test case validates the core voting functionality of the smart contract. The function `vote(0)` was executed successfully, and the transaction was mined on the Ethereum Sepolia network. This confirms that the system correctly records votes and updates the blockchain state without errors.

Test 2 – Duplicate Voting Prevention

Second vote attempt:

`vote(2)`

Output:

`revert: You already voted`

This confirms the security logic is working correctly.

The second test evaluates the security mechanism for preventing multiple votes from the same wallet address. When a second voting attempt was made using `vote(2)`, the transaction was rejected by the smart contract. The system returned a revert message stating “You already voted,” confirming that the one-vote-per-user rule is properly enforced.

Test 3 – Candidate Retrieval

`getCandidate(0)`

Output:

`("Ali", 1)`

This confirms vote count updates correctly.

The third test verifies the correctness of the data retrieval functions. The `getCandidate(0)` function was called to retrieve candidate information from the blockchain. The returned result `("Ali", 1)` confirms that the vote count was accurately updated and stored on-chain after the voting process.

3.5 Results

The smart contract was successfully deployed and tested on the Ethereum Sepolia test network. All core functionalities were verified through multiple test scenarios.

Voting transactions were executed successfully, and each vote was correctly recorded on the blockchain. The system effectively prevented duplicate voting attempts, as repeated voting from the same address resulted in transaction reversion. Additionally, candidate data retrieval functions returned accurate vote counts, confirming correct state updates.

These results demonstrate that the contract operates as intended and that the implementation is both reliable and functionally correct.

The decentralized voting system was successfully deployed and tested on the Ethereum Sepolia test network using Remix IDE and MetaMask. The testing phase focused on verifying the correct functionality of all smart contract components, including candidate initialization, voting logic, access control, and result retrieval.

First, the smart contract was deployed successfully without errors. During deployment, the initial set of candidates (such as Ali, Reza, and Sara) was properly initialized through the constructor function. The deployment process confirmed the correct integration between Remix IDE, MetaMask, and the Sepolia test network, where the transaction was successfully signed and recorded on-chain.

Next, the **addCandidate()** function was tested to evaluate the access control mechanism. A new candidate was added successfully by the contract owner, demonstrating that only authorized accounts are allowed to modify candidate data. This confirms that the owner-based restriction is functioning correctly and protecting sensitive operations within the contract.

The core functionality of the system, the **vote() function**, was then tested. Users were able to cast votes by selecting a candidate index, and each vote was successfully recorded on the blockchain. After each transaction, the vote count of the corresponding candidate was updated correctly, confirming that state changes were properly handled by the smart contract.

To validate the security mechanism, a duplicate voting attempt was performed using the same wallet address. The system correctly rejected the transaction and prevented the second vote from being recorded. This confirms that the one-vote-per-address rule is effectively enforced using the mapping-based tracking system, ensuring fairness in the voting process.

Finally, the **getCandidate()** and **getCandidatesCount()** functions were tested to verify data retrieval from the blockchain. The returned results accurately reflected the updated state of the system, including correct candidate names and vote counts. This demonstrates that the smart contract maintains consistent and reliable on-chain data storage.

Overall, all functionalities of the system operated successfully during testing. The results confirm that the smart contract is secure, reliable, and functions as intended. The system effectively demonstrates key blockchain principles, including transparency, immutability, and decentralized execution.

4. Business & Marketing Strategy

4.1 Target Market

Potential users include:

- Universities (student elections)
- Organizations (internal voting systems)
- Blockchain communities and DAOs

The decentralized voting system has potential applications across multiple sectors where transparency, security, and trust are critical requirements. One of the primary target markets includes educational institutions such as universities and colleges. In these environments, the

system can be used for student elections, club voting, and academic decision-making processes, where fairness and transparency are essential.

Another important target market consists of organizations and enterprises that require secure internal voting mechanisms. This includes corporate governance decisions, employee polling systems, and internal committee elections. The blockchain-based structure ensures that all votes are recorded immutably and cannot be altered after submission, increasing trust in organizational decision-making processes.

Additionally, blockchain communities and decentralized autonomous organizations (DAOs) represent a significant user base. These users are already familiar with blockchain technology and can benefit from a lightweight and transparent voting system that does not require complex token-based governance models. The simplicity of this system makes it suitable for both beginners and experienced blockchain users.

Overall, the target market includes any environment where transparent, tamper-proof, and verifiable voting is required, particularly where traditional centralized systems may lack trust or auditability.

4.2 Monetization Strategy

Possible revenue models include:

- Software-as-a-Service (SaaS) voting platform
- Transaction-based fees
- Enterprise blockchain integration

Although this project is implemented as a non-commercial academic prototype, several potential monetization strategies can be identified for future development and real-world deployment. One possible model is a Software-as-a-Service (SaaS) platform, where organizations subscribe to use a blockchain-based voting system hosted on a web interface. This would allow users to create elections, manage participants, and view results without directly interacting with smart contract code.

Another revenue model includes transaction-based fees, where a small fee is applied to each voting transaction or election deployment. This model aligns with existing blockchain economic structures where computational resources are compensated through gas fees and service charges. In addition, enterprise blockchain integration represents a high-value opportunity. Large organizations or governments could integrate the system into their existing infrastructure for secure decision-making processes. Customization, private blockchain deployment, and advanced features such as identity verification could be offered as premium services.

These monetization strategies demonstrate that blockchain-based voting systems have potential not only as academic projects but also as scalable commercial solutions.

4.3 Competitive Analysis

When compared to traditional centralized voting systems, blockchain-based voting provides significant advantages in terms of transparency, security, and immutability. Traditional systems often rely on centralized databases, which can be vulnerable to manipulation, data breaches, or

lack of auditability. In contrast, blockchain ensures that every vote is permanently recorded and publicly verifiable.

However, some existing blockchain-based voting solutions, particularly those based on DAO governance frameworks, offer highly complex mechanisms involving token-based voting, staking, and advanced governance logic. While these systems are powerful, they often introduce unnecessary complexity for simple voting use cases and can be difficult for new users to understand.

This project differentiates itself by focusing on simplicity, clarity, and educational value. It provides a minimal yet functional implementation of a decentralized voting system using Solidity smart contracts. By avoiding complex tokenomics and focusing on core functionalities such as one-vote-per-address enforcement and transparent result tracking, the system becomes more accessible and easier to reproduce.

Therefore, the proposed system positions itself between traditional centralized voting platforms and complex decentralized governance systems, offering a balanced solution that emphasizes both usability and blockchain principles.

5. Conclusion

This project successfully implements a decentralized blockchain-based voting system using Solidity. The final solution ensures:

- transparency
- immutability
- prevention of duplicate voting
- secure smart contract interaction

The project successfully demonstrated the design and implementation of a decentralized voting system using smart contracts on the Ethereum Sepolia test network. The system was developed based on foundational blockchain principles introduced in works such as Nakamoto (2008) and Szabo (1997), which emphasize decentralization, trust minimization, and data integrity. Using tools such as Solidity, Remix IDE, and MetaMask, the system was implemented and tested in a real blockchain environment.

5.1 Key Findings

- Blockchain ensures tamper-proof data
- Smart contracts automate trust
- Decentralization removes reliance on central authorities

The results of this project confirm that blockchain technology can effectively support secure and transparent voting systems. The implemented smart contract successfully enforces the one-vote-per-user rule by tracking wallet addresses, preventing duplicate voting attempts. In addition, all votes are permanently recorded on the blockchain, ensuring immutability and auditability of election results. The use of access control mechanisms guarantees that only authorized users can modify candidate data, improving system security. Furthermore, testing on the Sepolia network

verified that all core functionalities, including voting, candidate management, and result retrieval, operate as expected without errors.

5.2 Future Work

- Develop a frontend user interface (DApp)
- Add multi-election support
- Integrate identity verification
- Implement token-based voting

Although the current system successfully implements a basic decentralized voting mechanism, several improvements can be made in future development. One key enhancement is the integration of decentralized identity (DID) systems to strengthen voter authentication beyond wallet-based identification. Additionally, a front-end decentralized application (DApp) can be developed to improve user experience and accessibility. Scalability improvements may also be required to support large-scale elections with a high number of participants. Furthermore, implementing protection against advanced blockchain attacks such as front-running could further enhance system security. Finally, expanding the system to support multiple elections and role-based voting could make it more suitable for real-world governance applications.

References (APA Style)

Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system.

Buterin, V. (2014). A next-generation smart contract and decentralized application platform (Ethereum White Paper).

Szabo, N. (1997). Formalizing and securing relationships on public networks. *First Monday*, 2(9).

Ethereum Foundation. (2023). Ethereum documentation. <https://ethereum.org/en/developers/docs/>

Antonopoulos, A. M., & Wood, G. (2018). *Mastering Ethereum: Building smart contracts and DApps*. O'Reilly Media.

Appendix

Include the following:

- Full code
- Smart Contract
- Deployment logs
- Public Key & Address